

oshizo / **sbert-jsnli-luke-japanese-base-lite**   like 35

 Sentence Similarity  sentence-transformers  PyTorch  Transformers

 shunk031/jsnli  Japanese luke feature-extraction  License: apache-2.0

 Train  Deploy  Use this model

 Model card  Files  xet  Community 1

Downloads last month

10,920



Inference Providers NEW

HF

 Sentence Similarity

Source Sentence

Your sentence here...

Sentences to compare to

Your sentence here...

Your sentence here...

Add Sentence

Generate

 View Code

 Dataset used to train oshizo/sbert-jsnli-luke-japanese-base-lite

 Spaces using oshizo/sbert-jsnli-luke-japanese-base-lite 3

 Collection including oshizo/sbert-jsnli-luke-japanese-base-lite

japanese-embedding-models Collection

3 items • Updated Feb 3, 2024 •  5

sbert-jsnli-luke-japanese-base-lite

This is a [sentence-transformers](#) model: It maps sentences & paragraphs to a 768 dimensional dense vector space and can be used for tasks like clustering or semantic search.

The base model is [studio-ousia/luke-japanese-base-lite](#) and was trained 1 epoch with [shunk031/jsnli](#).

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

oshizo/sbert-jsnli-luke-japanese-base-lite is supported by the following Inference Providers:

 HF Inference API

 View API Code

Dismiss

🔗 Usage (Sentence-Transformers)

Using this model becomes easy when you have [sentence-transformers](#) installed:

```
pip install -U sentence-transformers
```

Then you can use the model like this:

```
from sentence_transformers import SentenceTransformer
sentences = ["This is an example sentence", "Each sentence is converted"]

model = SentenceTransformer('oshizo/sbert-jsnli-luke-japanese-base-lite')
embeddings = model.encode(sentences)
print(embeddings)
```

🔗 Usage (HuggingFace Transformers)

Without [sentence-transformers](#), you can use the model like this: First, you pass your input through the transformer model, then you have to apply the right pooling-operation on-top of the contextualized word embeddings.

```
from transformers import AutoTokenizer, AutoModel
import torch

#Mean Pooling - Take attention mask into account for correct averaging
def mean_pooling(model_output, attention_mask):
    token_embeddings = model_output[0] #First element of model_output contains all token embeddings
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_expanded.sum(1), min=1e-9)

# Sentences we want sentence embeddings for
sentences = ['This is an example sentence', 'Each sentence is converted']

# Load model from HuggingFace Hub
tokenizer = AutoTokenizer.from_pretrained('oshizo/sbert-jsnli-luke-japanese-base-lite')
model = AutoModel.from_pretrained('oshizo/sbert-jsnli-luke-japanese-base-lite')

# Tokenize sentences
encoded_input = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')

# Compute token embeddings
with torch.no_grad():
    model_output = model(**encoded_input)
```

```
# Perform pooling. In this case, mean pooling.
sentence_embeddings = mean_pooling(model_output, encoded_input['attention_mask'])

print("Sentence embeddings:")
print(sentence_embeddings)
```

[🔗](#) Evaluation Results

The results of the evaluation by JSTS and JSICK are available [here](#).

[🔗](#) Training

Training scripts are available in [this repository](#). This model was trained 1 epoch on Google Colab Pro A100 and took approximately 40 minutes.